# Intitulé du cours:
# Technique des applications web

## Chapitre 1 :
## Introduction à React et mise en place de l'environnement

Filière: 2éme année – bts web

**Enseignante:** Abid Arij

Année de formation 2025-2026

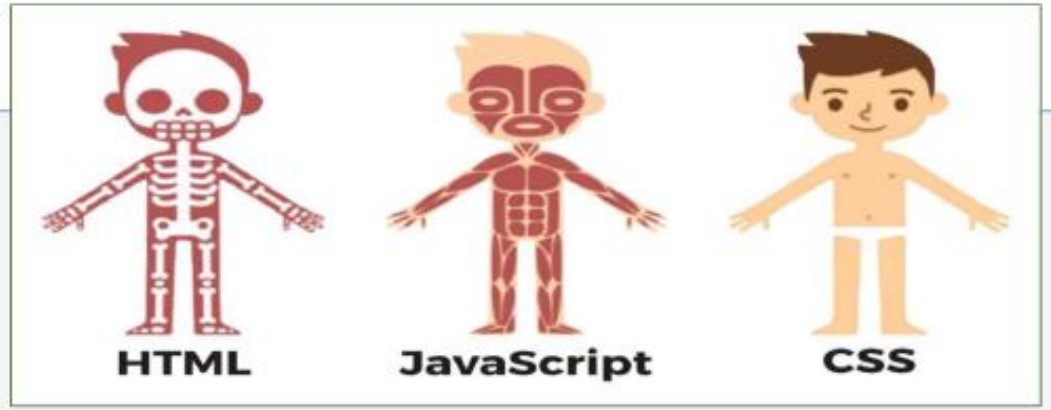
React JS

# Plan

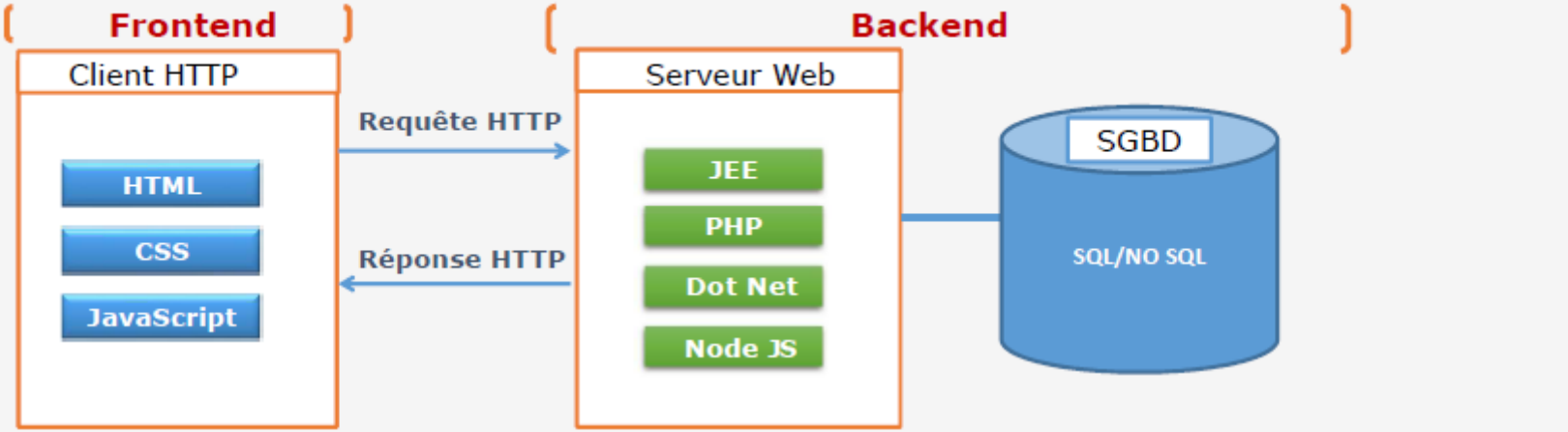
- Technologies web
- Les frameworks JavaScript
- Rappels : JavaScript ES2015 (ES6)
- Initiation à JSX
- Présentation générale du ReactJS

# Technologies web

- **HTML** : **HyperText Markup Language** (langage de balisage hypertexte ou **HTML**) est le langage utilisé pour décrire et définir le contenu d'une page web.
- **CSS** : **Cascading Style Sheets** (feuilles de style en cascade ou **CSS**) est utilisé pour décrire l'apparence du contenu d'une page web.
- **JavaScript:** c'est un langage de programmation de scripts principalement employé dans les pages web interactives (mais aussi pour les serveurs avec l'utilisation de **Node.js**).



# Technologies Web

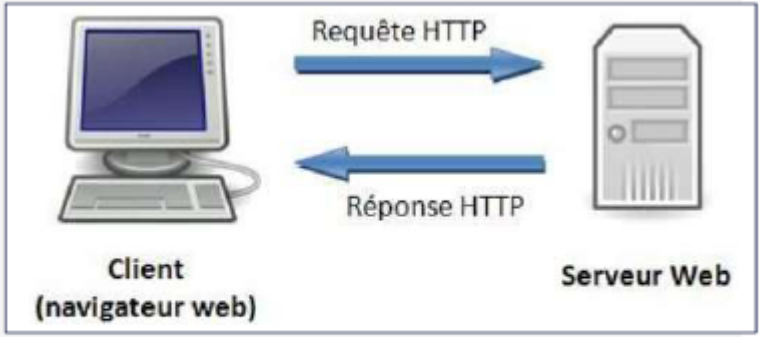



- Un client web (Browser) communique avec le serveur web en utilisant le protocole **HTTP**
- Une application web se compose de deux parties:
  - La partie **Frontend** : S'occupe de la présentation des IHM côté Client :
    - Langages utilisés : HTML, CSS, Java Script
  - La partie **Backend** : S'occupe des traitements effectués côté serveur :
    - Technologies utilisées : PHP, JEE, .Net, Node JS
- La communication entre la partie **Frontend** et la partie **Backend** se fait en utilisant le protocole **HTTP**

# Le protocole HTTP

- **HTTP : Hypertext Transfer Protocol (HTTP)** est un protocole de la couche application, orienté client-serveur, qui permet le transfert de documents web tels que des documents HTML, XML et JSON.
- Une requête HTTP peut être envoyée en utilisant les méthodes suivantes :

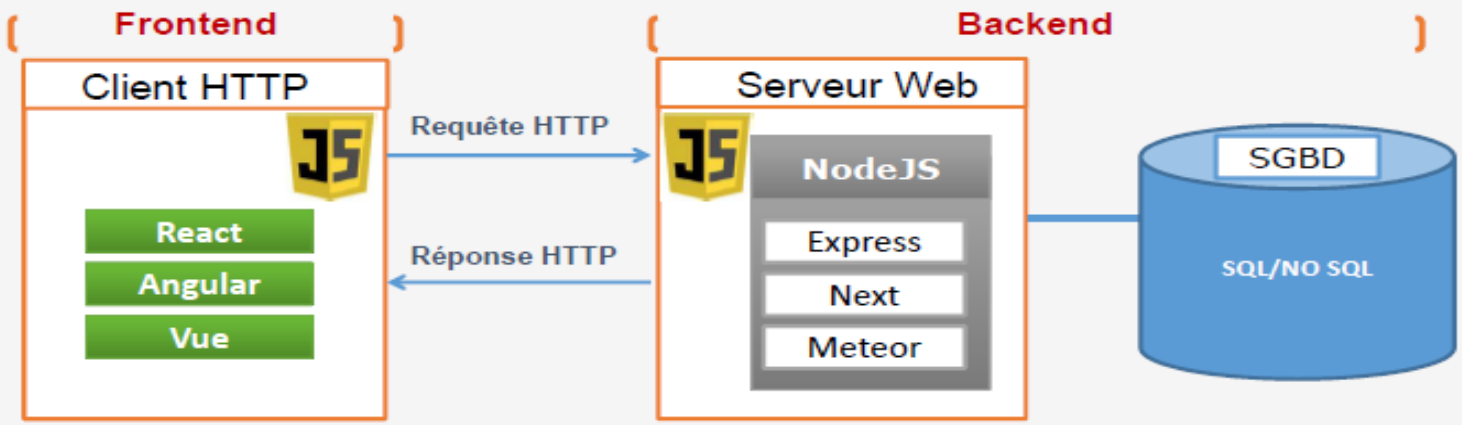
- GET : Pour récupérer le contenu d'un document
- POST : Pour soumissionner des formulaires (Envoyer, dans la requête, des données saisies par l'utilisateur )
- PUT pour envoyer un fichier du client vers le serveur
- DELETE permet de demander au serveur de supprimer un document
- HEAD permet de récupérer les informations sur un document (Type, Capacité, Date de dernière modification etc…)



# Technologies Web

- **Notion de framework** :
  - Plusieurs traductions : cadriciel, environnement de développement, cadre d'applications, ...
  - Ensemble de composants logiciels
  - Facilitant la réalisation d'une (partie de l') application
  - Imposant une certaine structure, logique, syntaxe...
- **Frameworks applicatifs pour le développement d'applications web :**
  - **Angular**, **React** pour JavaScript,
  - **Spring** pour Java,
  - **Symfony** pour PHP, ...
- **Frameworks de présentation de contenu web : Bootstrap** pour CSS, ...
- **Frameworks de persistance de données : hibernate, ...**
- **Frameworks de logging : log4j, ...**

# Technologies web




- Framework Frontend
  - ReactJS (2013),
  - AngularJS (2009),
  - VueJS (2014),
  - EmberJS (2011),
  - PolymerJS (2015)
  - …

- Framework Backend
  - ExpressJS (2010),
  - NextJS (2018),
  - MeteorJS (2012),
  - …

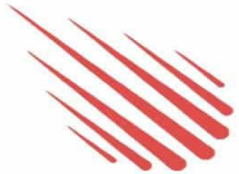
# Les frameworks javaScript : évolution

- Evolution JavaScript :

# Le langage javaScript

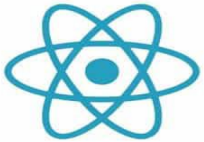
- **ECMAScript :**
  - Ensemble de normes standardisées par l'organisation européenne Ecma International pour l'utilisation des langages de programmation du type Script.

# JavaScript côté client



Meteor JS

Ember JS



React JS

Vue JS

Polymer JS

Angular JS

# Rappel : JavaScript ES2015 (ES6)

- **Déclaration et portée des variables :**
  - **let, const :** définir des variables et des constantes
- **Structures conditionnelles :**
  - **if ... else if ... else**
  - **variable = condition ? expressionIf : expressionElse;**
- **Structures itératives :**
  - **for .. in :** avec les objets - `for (let key in object)`
  - **for .. of :** avec les tableaux - `for (let v of array)`
- **Fonctions :**
  - **Fonctions nommées :** `function sum(n1, n2)`
  - **Fonctions anonymes :** `let sum = function (n1, n2)`
  - **Fonctions fléchées :** `let f = (name) => "hello" + name;`

# Rappel : JavaScript ES2015 (ES6)

## Fonction flechées (arrow functions)

```
function addn(a,b)
{

    return a+b;
}

console.log(addn(5,8));
```

```
const add=(a,b)=>{


    return a+b;
}

console.log(add(1,3));
```

```
const add=(a,b)=> a+b;
```

```
const pow=a=> a*a;
```

# Rappel : JavaScript ES2015 (ES6)

- **Méthodes de manipulation des tableaux :**
  - Remplir un tableau : **fill**
  - Inverser et trier un tableau : **reverse** et **sort**
  - Ajouter des éléments en queue du tableau : **concat**
  - Extraire un sous-tableau : **slice**
  - Trouver le premier et le dernier indice d'une valeur :
    - **indexOf**
    - **lastIndexOf**

```
let t = [0, 1, 2, 3, 4];
t.fill(0); // [0, 0, 0, 0, 0]

let t = [0, 1, 2, 3, 4];
t.reverse(); // [4, 3, 2, 1, 0]
t.sort(); // [0, 1, 2, 3, 4]

let t = [1, 2, 3];
t.concat(4, 5); // [1, 2, 3, 4, 5]

let t = [1, 2, 3, 4, 5];
t.slice(1, 4); // [2, 3, 4]
t.slice(2); // [3, 4, 5]
t.slice(); // [1, 2, 3, 4, 5]

let t = [1, 2, 3, 4, 3, 5, 3];
t.indexOf(3); // 2
t.lastIndexOf(3); // 6
```

# Rappel : JavaScript ES2015 (ES6)

- **Méthodes de manipulation des tableaux :**
- Ajouter/supprimer un élément :
  - en queue du tableau : **push** et **pop**
  - en tête du tableau : **unshift** et **shift**
  - à partir de n'importe quelle position : **splice**

```
let t = [0, 1, 2, 3, 4];
t.pop();  // 4 , t = [0, 1, 2, 3]
t.shift(); // 0, t = [1, 2, 3]
t.unshift(-1); // 4, t = [-1, 1, 2, 3]

let t = [1, 2, 3, 4, 5];
t.splice(3,1); // 4
--supprime 1 elt à partir de l'indice 3
// t = [1, 2, 3, 5]
```

```
const months= ['Jan','March','April','June'];
months.splice(1, 0, 'Feb');
-- ajoute à l'indice 1 Feb
// ["Jan", "Feb", "March", "April", "June"]

months.splice(4, 1, 'May'); // ["June"]
-- remplace 1 élément à l'indice 4 par May
// ["Jan", "Feb", "March", "April", "May"]
```

# Rappel : JavaScript ES2015 (ES6)

- **Map :**
  - Parcourir les éléments d'un tableau.
- **Reduce :**
  - Combiner les valeurs : réduire les valeurs en une seule valeur
- **Filter :**
  - Filtrer les éléments d'un tableau suivant une certaine condition.

```
let jeux = ['go', 'de'];
let prix = [99, 15];
let t = jeux.map((j, i) => ({ nom: j, prix: prix[i]
}));
// [{ nom: 'go', prix: 99 }, { nom: 'de', prix: 15 }]
```

```
let t = [0, 1, 2, 3];
let somme = t.reduce((a, b) => a + b);
console.log("Somme=" + somme);
```

```
let t3 = t.filter((valeur) => {
    return valeur < 8;
});
```

```
let pets = ['cat', 'dog', 'fish'];
let nb = pets.map(s => s.length).reduce((a, b) => a + b);
console.log('nb = ' + nb);
```

# Rappel : JavaScript ES2015 (ES6)

## Spread (tableaux et objets)

```
const arr = [1,2,3];
const arr2 = [...arr, 4]

console.log(arr2);
```

```
const myObj = {
    a: 1,
    b: 2,
    c: 3
}
const myObj2 = {
   ...myObj,
    d: 4
}

console.log(myObj2);
```

# Introduction à JSX

- **JSX (JavaScript XML)** est une **syntaxe** utilisée dans **React** qui permet d'écrire du code **qui ressemble à du HTML à l'intérieur du JavaScript**.
- ⚠️ Point crucial (à ne jamais oublier) :
  👉 Ce n'est **NI du HTML pur**, **NI du JavaScript pur**
  ➡️ C'est un **mélange des deux**
  👉 **JSX n'est PAS compris par le navigateur**
- Le navigateur ne comprend que **JavaScript pur**.

```
const element = <h1>Bonjour React</h1>;
```

Ce code **n'est pas valide en JavaScript pur**.

# À quoi sert JSX ?

JSX sert à :

- Décrire **l'interface utilisateur (UI)**
- Dire à React **quoi afficher à l'écran**
- Écrire le code **plus lisible et plus clair**

**Sans JSX,** écrire React serait **beaucoup plus compliqué**.

❌ **Sans JSX (JavaScript pur)**

```
const element = React.createElement(
  "h1",
  null,
  "Bonjour React"
);
```

👉 Difficile à lire 😵

✅ **Avec JSX**

```
const element = <h1>Bonjour React</h1>;
```

- **Plus simple**
- **Ressemble au HTML**
- **Plus rapide à écrire**

💡 React **n'oblige pas** à utiliser JSX, mais **99 % des développeurs l'utilisent**.

# JSX

- **Déclaration des variables et constantes**

```
const PI = <h1>3.14</h1>;
var name = 'Tout le monde';
```

- **Utiliser des expressions dans JSX**

```
const name = 'Tout le monde';
const element = <h1 count={1 + 2 + 3 + 4} >Bonjour, {name}</h1>;
```

- **Les propriétés :** On ne parle plus d'attributs HTML mais de propriétés d'un composant : Chaque balise HTML est en réalité un composant à part entière qui accepte des propriétés dites : **props**.

```
<input
   type="number" // String
   max={100} // La valeur 100
   onChange={this.handleChange} // Fonction
/>
```

# Différence entre JSX / HTML / JavaScript

- **Différence 1 : JSX est du JavaScript**

En JSX, tu peux utiliser **des variables JS** :

```
const name = "AriJ";
```

<h1>Bonjour {name}</h1>

{} permet d'insérer du JavaScript dans JSX

➡ Impossible en HTML pur

- **Différence 2 HTML VS JSX :**

| HTML | JSX |
|---|---|
| for | htmlFor |
| class | className |
| <style color ="blue"> | < style={{color:'blue'}}> |
| < !--Comment--> | { /*Comment*/ } |

# Différence entre JSX JavaScript

| JavaScript | JSX |
|---|---|
| Langage de programmation | Extension de JavaScript |
| Logique, calculs | Affichage UI |
| Compris par le navigateur | Doit être compilé |
| Pas de balises HTML | Utilise des balises |

# JS VS JSX

- **JSX :** Souplesse dans l'écriture, la lecture et la compréhension du code.

**Sans JSX ☹**

```
React.createElement(
    'div',
    null,
    React.createElement('h2', null, 'Title List'),
    React.createElement('p', null, 'Contenu')
)
```

**Avec JSX ☺**

```
const element = () => (
    <div>
        <h2>Title List</h2>
        <p>Contenu</p>
    </div>
```

# HTML VS JSX

- **Convertir en ligne HTML en JSX :**
  - https://magic.reactjs.net/htmltojsx.html

## HTML to JSX Compiler

☑ Create class · Class name: NewComponent

Live HTML Editor

```
<!-- Hello world -->
<div class="awesome" style="border: 1px solid red">
  <label for="name">Enter your name: </label>
  <input type="text" id="name" />
</div>
<p>Enter your HTML here</p>
```

```
var NewComponent = React.createClass({
  render: function() {
    return (
      <div>
        {/* Hello world */}
        <div className="awesome" style={{border: '1px solid red'}}>
          <label htmlFor="name">Enter your name: </label>
          <input type="text" id="name" />
        </div>
        <p>Enter your HTML here</p>
      </div>
    );
  }
});
```

# JS VS JSX

- **Convertir en ligne JSX en JS :**
  - https://infoheap.com/online-react-jsx-to-javascript/
  - https://babeljs.io/

**Input jsx**

example:

**ReactDOM.render(<h1>Hello world</h1>, document.getElementById('hello'));**

```
ReactDOM.render(
   <h1>Hello world</h1>,
document.getElementById('hello'));
```

convert above jsx to javascript (char count: 77)

jsx is valid (input char count: 72)

**Converted javascript**

```
ReactDOM.render(React.createElement(
  'h1',
  null,
  'Hello world'
), document.getElementById('hello'));
```

# JSX est-il compris par le navigateur ?

❌ **NON**

Le navigateur **ne comprend pas JSX** 😮

- JSX doit être **transformé (compilé)** en JavaScript classique
- C'est **Babel** qui fait ce travail

Le navigateur comprend **SEULEMENT JavaScript**

Donc :

➡️ JSX est **transformé** en JavaScript par **Babel**

<h1>Hello</h1>

⬇️ devient :

React.createElement("h1", null, "Hello");

# Babel

Babel est un **transpileur** ECMAScript 6.

- Il **transforme** du code JavaScript **moderne** (ES6+, JSX) en **JavaScript standard** que **les navigateurs comprennent**
- **Babel ne compile PAS comme C ou Java**
  Il **réécrit** le code → on parle de **transpilation**, pas de compilation native.

**Code écrit par le développeur (JSX + ES6)**

```
const App = () => <h1>Hello</h1>;
```

**Code après Babel**

```
const App = function () {
  return React.createElement("h1", null, "Hello");
};
```

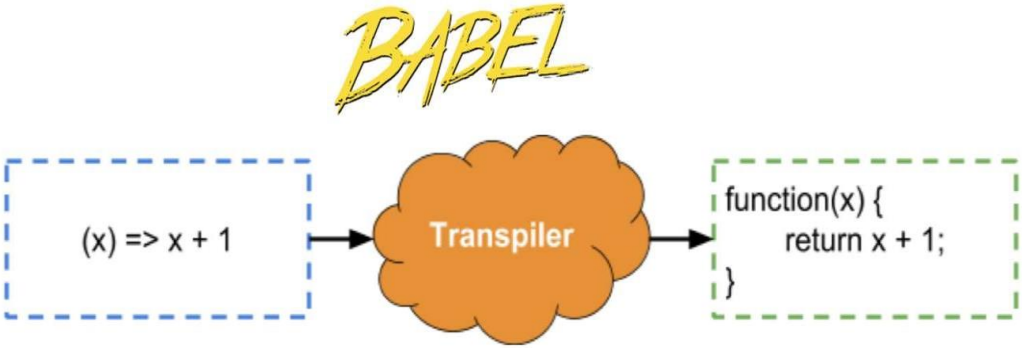
**Pourquoi Babel est indispensable en React ?**

Le navigateur :

- ❌ ne comprend PAS JSX
- ❌ ne comprend PAS toujours ES6+ (`arrow function`, `import`, etc.)

React **utilise JSX par défaut**

👉 **Sans Babel → l'application ne démarre pas**



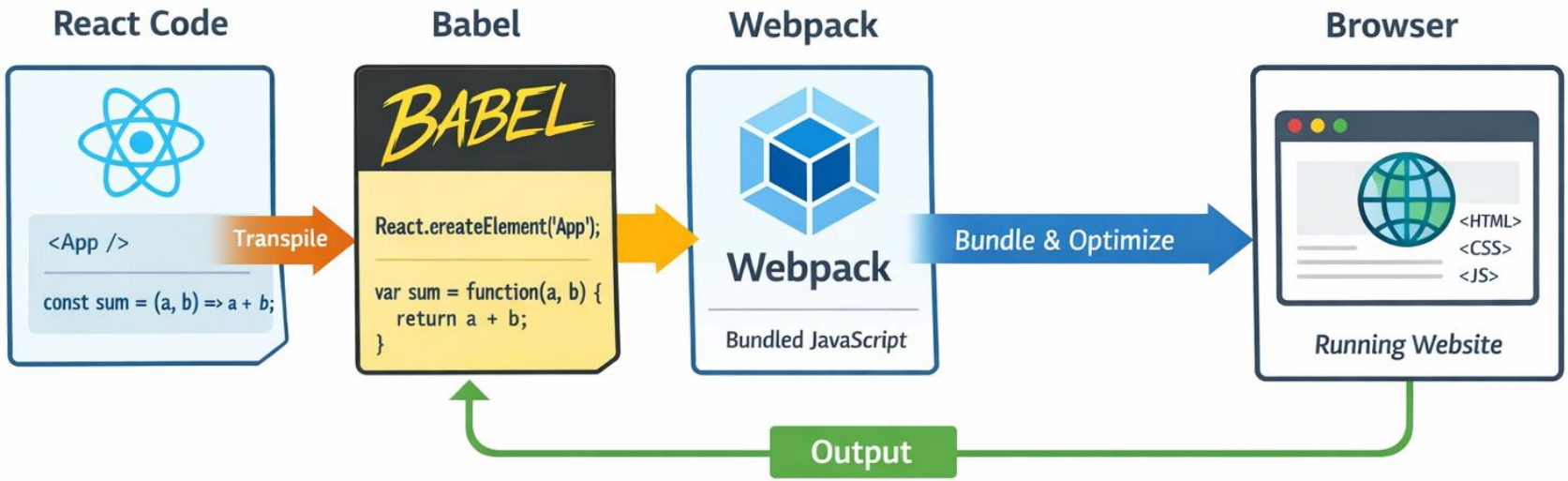
# Babel : transforme quoi exactement ?

- En environnement **React**, **Babel** joue un rôle fondamental : Babel transforme le JSX en JavaScript standard

| Avant | Après |
|---|---|
| JSX | React.createElement() |
| let / const | var |
| Arrow functions | Functions classiques |
| Optional chaining | Code compatible |
| ES6+ | ES5 |

# Webpack react

- Processus React → Babel → Webpack → Navigateur
- Ce pipeline correspond au **cycle de compilation et d'exécution d'une application React moderne**.



# Webpack (Bundling)

- ❑ Webpack est un **outil de bundling (regroupement de modules)** essentiel dans l'écosystème React moderne. Pour bien comprendre, il faut voir React non pas comme une simple bibliothèque UI, mais comme un projet composé de **plusieurs fichiers interconnectés** (JS, CSS, images, etc.).

- ❑ Regrouper les fichiers (Bundling) :Analyse les imports (import/export)

Regroupe tous les fichiers JS en **un seul fichier bundle**

Webpack :

- Analyse tous les import
- Construit un **graphe de dépendances**
- Regroupe tout dans un fichier final (bundle.js)

webpack

# Webpack (Bundling)

❑ **Gérer les assets**

Webpack peut aussi gérer :

- CSS
- Images
- Fonts

❑ **Optimiser le code**

Webpack permet :

- Minification (réduction taille)
- Code splitting (chargement à la demand
- Tree shaking (suppression code inutile)

# Comment fonctionne Webpack (étapes internes)

- **Étape 1 : Point d'entrée (Entry)**

Webpack commence par un fichier principal :

entry: "./src/index.js"

➡ C'est le point de départ de l'analyse

- **Étape 2 : Graphe de dépendances**

Webpack lit tous les imports :

```
import App from "./App";
import "./style.css";
```

➡ Il construit un **dependency graph** (graphe de dépendances)

# Comment fonctionne Webpack (étapes internes)

- **Étape 3 : Loaders (transformations)**

Les loaders permettent de transformer les fichiers :

| Type | Loader |
|---|---|
| JSX | babel-loader |
| CSS | css-loader |
| Images | file-loader |

**Exemple :**

```
module: {
  rules: [
    {
      test: /\.js$/,
      use: "babel-loader"
    }
  ]
}
```

- Dans ce cas, après webpack ou **npm run build**, tu trouveras **bundle.js dans le dossier dist/** :

```
projet/
 ├─ src/
 │   ├─ index.js
 │   └─ App.js
 ├─ dist/
 │   └─ bundle.js   <-- ici !
 └─ webpack.config.js
```

Le **build** est le **processus final** où ton projet React passe de “code source pour développeur” → “code optimisé pour le navigateur”.

# Pourquoi on fait un build

**Ce qui se passe pendant le build :**

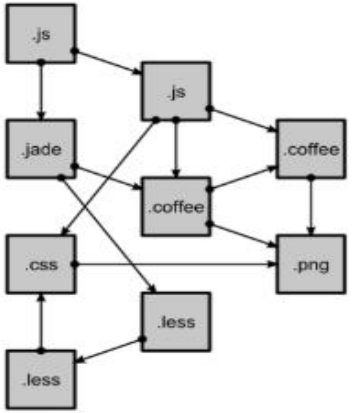
- Webpack lit tous tes fichiers (src/)
- Transforme JSX, ES6 → JS standard
- Regroupe tous les fichiers → crée bundle.js (ou équivalent)
- Optimise le code (minification, hash, tree-shaking)
- Place le résultat dans un dossier build/ ou dist/ prêt à être déployé

En résumé : **Webpack est l'outil qui fait le build.**
**Le build est le résultat optimisé prêt pour la production.**

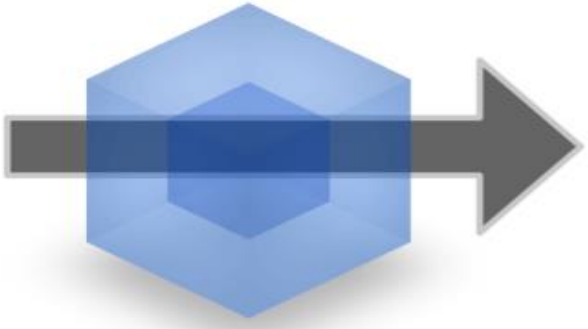
```
build/
 ├─ index.html
 ├─ assets/
 │   ├─ main.abc123.js    <-- ton bundle
optimisé
 │   └─ main.abc123.css
```

# Webpack (Bundling)

**Avant :**

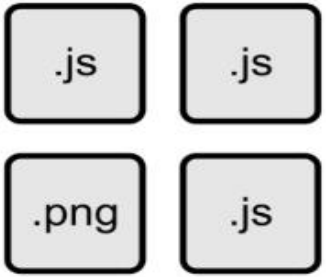



modules
with dependencies



webpack
MODULE BUNDLER

**Après Webpack :**




static
assets

=> Produire un fichier optimisé prêt pour la production.

# Merci pour

# votre Attetion